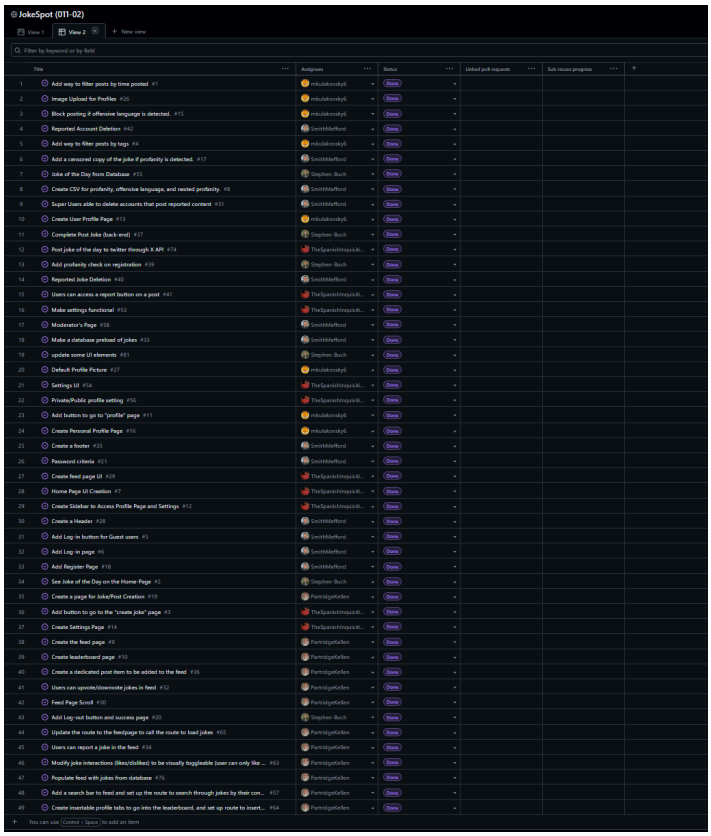
Title:
JokeSpot

Who (Name – GitHub):
Kellen Partridge – PartridgeKellen
Sam Shoemaker – TheSpanishInquisition
Smith Mefford – SmithMefford
Michael Kulakovsky – mkulakovsky6
Stephen Buch – Stephen-Buch

Project Description:

JokeSpot is a light-weight social media platform where users can view, share, and tag their jokes to the site. This platform primarily focuses on the collection of text based jokes, and collects minimal user data as that was a core ideology of the development team. Jokespot uses a very comedy forward UI, a database collects all jokes and shares the content to the feed page of the application. Registered users are welcome to create their own jokes which will be appended to the database and will be shown at the top of the feed. This platform also features a leaderboard and joke of the day to celebrate users who have the best jokes and interact with the site the most often. JokeSpot is made for those who want to get their dose of laughter for the day without the drama, politics, and current events dragging them down.

Project Tracker



JokeSpot (011-02)

View 1 | View 2 | + New view

Filter by keyword or by field

	Title	Assignee	Status	Linked pull requests	Sub-issues progress
1	Add way to filter posts by time posted #1	mkulakovsky6	Done		
2	Image Upload for Profiles #26	mkulakovsky6	Done		
3	Block posting if offensive language is detected. #15	mkulakovsky6	Done		
4	Reported Account Deletion #42	SmithMefford	Done		
5	Add way to filter posts by tags #4	mkulakovsky6	Done		
6	Add a censored copy of the joke if profanity is detected. #17	SmithMefford	Done		
7	Joke of the Day from Database #55	Stephen-Buch	Done		
8	Create CSV for profanity, offensive language, and nested profanity. #8	SmithMefford	Done		
9	Super Users able to delete accounts that post reported content #31	SmithMefford	Done		
10	Create User Profile Page #13	mkulakovsky6	Done		
11	Complete Post Joke (back-end) #37	Stephen-Buch	Done		
12	Post joke of the day to twitter through X API #74	TheSpanishInquisiti...	Done		
13	Add profanity check on registration #39	Stephen-Buch	Done		
14	Reported Joke Deletion #40	SmithMefford	Done		
15	Users can access a report button on a post #41	TheSpanishInquisiti...	Done		
16	Make settings functional #53	TheSpanishInquisiti...	Done		
17	Moderator's Page #38	SmithMefford	Done		
18	Make a database preload of jokes #33	SmithMefford	Done		
19	update some UI elements #81	Stephen-Buch	Done		
20	Default Profile Picture #27	mkulakovsky6	Done		
21	Settings UI #54	TheSpanishInquisiti...	Done		
22	Private/Public profile setting #56	TheSpanishInquisiti...	Done		
23	Add button to go to "profile" page #11	mkulakovsky6	Done		
24	Create Personal Profile Page #16	mkulakovsky6	Done		
25	Create a footer #35	SmithMefford	Done		
26	Password criteria #21	SmithMefford	Done		
27	Create feed page UI #29	TheSpanishInquisiti...	Done		
28	Home Page UI Creation #7	TheSpanishInquisiti...	Done		
29	Create Sidebar to Access Profile Page and Settings #12	TheSpanishInquisiti...	Done		
30	Create a Header #28	SmithMefford	Done		
31	Add Log-in button for Guest users #5	SmithMefford	Done		
32	Add Log-in page #6	SmithMefford	Done		
33	Add Register Page #18	SmithMefford	Done		
34	See Joke of the Day on the Home-Page #2	Stephen-Buch	Done		
35	Create a page for Joke/Post Creation #19	PartridgeKellen	Done		
36	Add button to go to the "create joke" page #3	TheSpanishInquisiti...	Done		
37	Create Settings Page #14	TheSpanishInquisiti...	Done		
38	Create the feed page #9	PartridgeKellen	Done		
39	Create leaderboard page #10	PartridgeKellen	Done		
40	Create a dedicated post item to be added to the feed #36	PartridgeKellen	Done		
41	Users can upvote/downvote jokes in feed #32	PartridgeKellen	Done		
42	Feed Page Scroll #30	PartridgeKellen	Done		
43	Add Log-out button and success page #20	Stephen-Buch	Done		
44	Update the route to the feed/page to call the route to load jokes #65	PartridgeKellen	Done		
45	Users can report a joke in the feed #34	PartridgeKellen	Done		
46	Modify joke interactions (likes/dislikes) to be visually toggleable (user can only like ... #63	PartridgeKellen	Done		
47	Populate feed with jokes from database #76	PartridgeKellen	Done		
48	Add a search bar to feed and set up the route to search through jokes by their con... #57	PartridgeKellen	Done		
49	Create insertable profile tabs to go into the leaderboard, and set up route to insert... #64	PartridgeKellen	Done		

+ You can use Control + Space to add an item

Project Board on GitHub: https://github.com/users/SmithMefford/projects/1

Video
Demo Video: 2026-04-22 13-22-50.mkv

VCS
Git Repository on GitHub: https://github.com/SmithMefford/JokeSpot

Contributions
Smith: Responsible for creating the login and register pages from UI to api routes and making sure the authentication middleware was functional. Designed the website's header and footer, ensuring there were links to our X account, and to the navbar and profile pages. Also made custom middleware for user0, that would let them access the admin panel which was also designed by me from front end to back end for functional deletion of jokes and accounts. Finally I worked on the filtering and censoring of profanity within jokes via a custom javascript along with .csv files that the jokes are checked against.

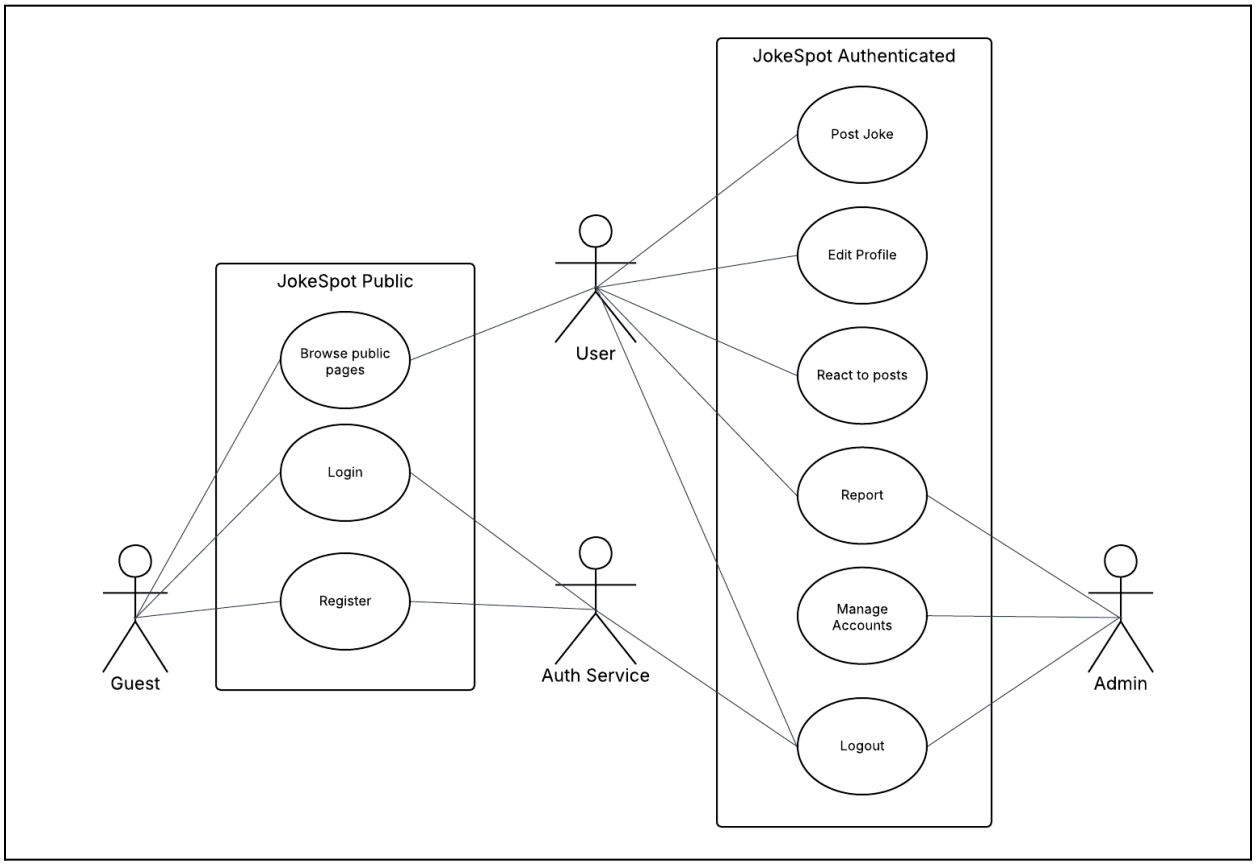
Sam: Responsible for creation of the sidebar elements, the entire settings page along with the functionality of the settings, specifically the dark mode, autorefresh, and the framework for the vulgarity filter. Additionally worked on UI elements for most of the entire site. Worked both in handlebars/html and javascript to ensure the functionality of buttons and routing through the site as well as middleware to make sure everything was running as expected

Michael: Responsible for the profile elements, hate-speech blocking, and filtering by time. Did both frontend and backend work, designing handlebars pages and adding image support to the database for custom profile pictures. Also made some general UI element changes, changing how time of posts were displayed as a date and time to how long ago they were posted. Implemented a filter into joke posting such that especially offensive language couldn't be posted.

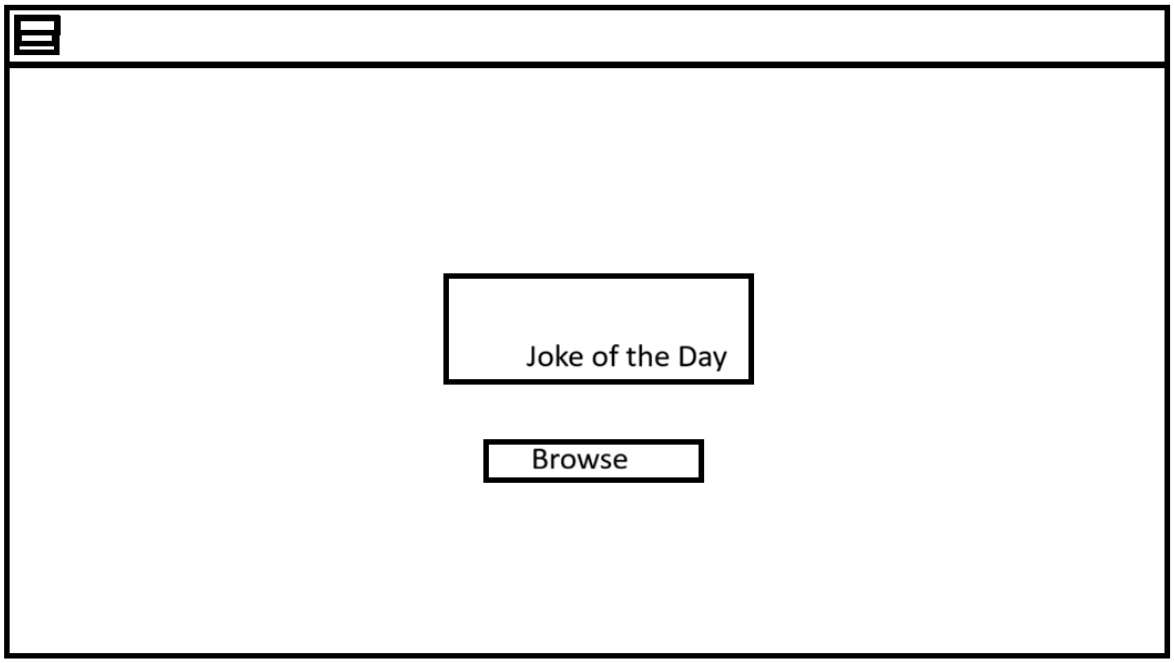
Kellen: Responsible for creating the feed page, the joke elements, and the leaderboard page. Handled populating the feed jokes from the database, as well as added a search bar to filter through jokes in the database. Also designed the joke elements used to populate the feed, displaying information about the joke including the author, timestamp, and content. Joke interactions were also added, those being the like/dislike functionality and the ability to report jokes. Also created the leaderboard page and the elements that populate it, displaying information about users.

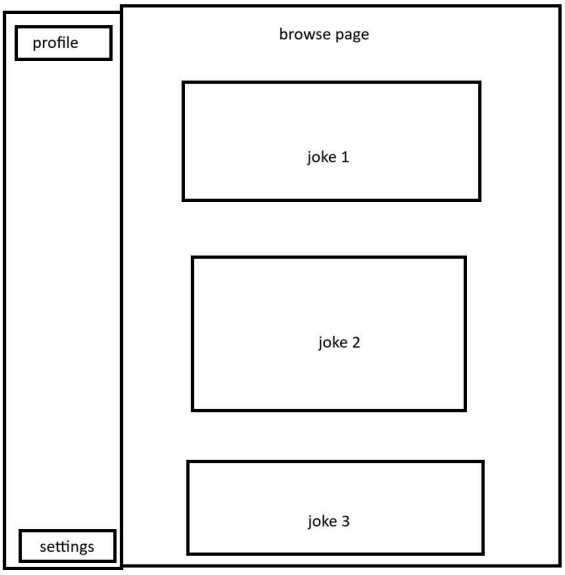
Stephen: Responsible for the 'Joke of the Day' functionality and homepage, logout route, posting jokes, implementing unit testing, and setting profanity filter on the register page. I also helped debug and smooth out different aspects of the application, such as improving dark mode UI and other visual elements in the navbar and settings page. Also helped set up the handlebars partials early on, made the site's logo, and reviewed sizable pull requests.

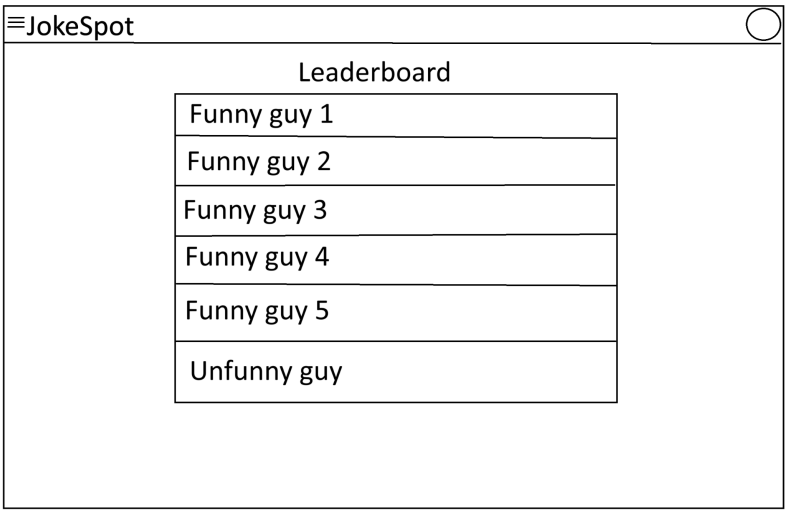
Use Case Diagram

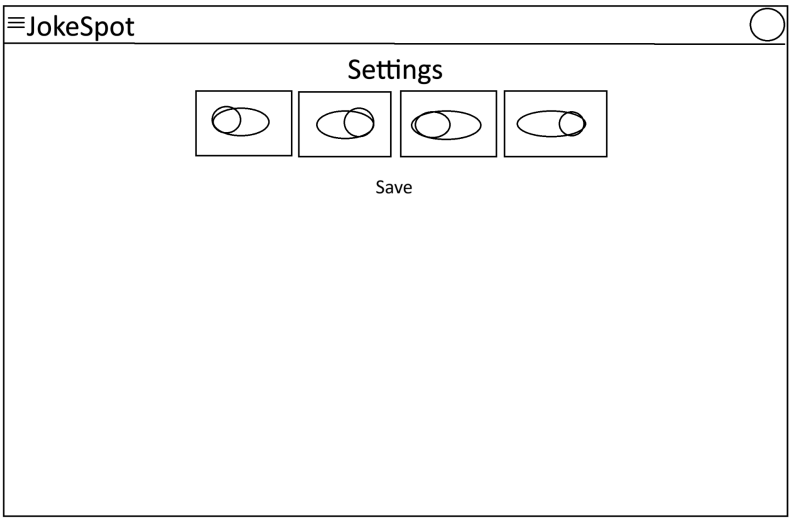


Wireframes:




profile
browse page
joke 1
joke 2
joke 3
settings


≡JokeSpot
Leaderboard
Funny guy 1
Funny guy 2
Funny guy 3
Funny guy 4
Funny guy 5
Unfunny guy


≡JokeSpot
Settings
Save

≡JokeSpot

Post Joke

text body

tags

Post

≡JokeSpot

User Name

Likes

Rank

#1

≡JokeSpot

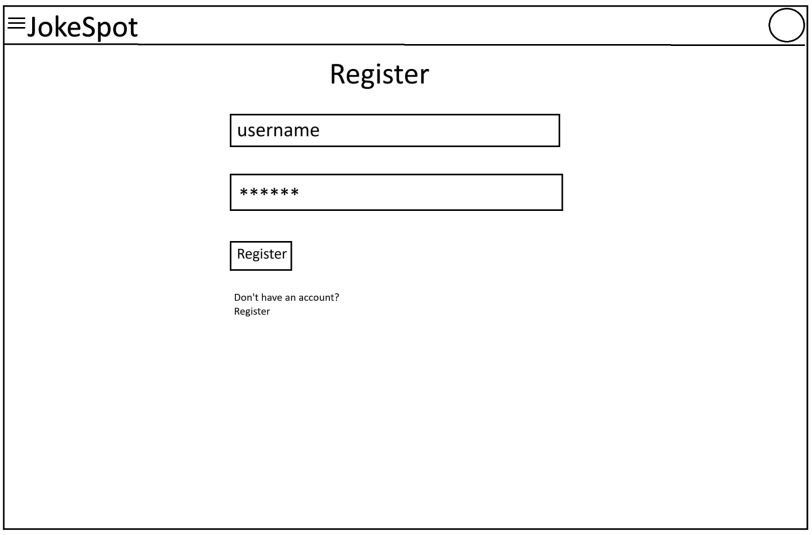
Login

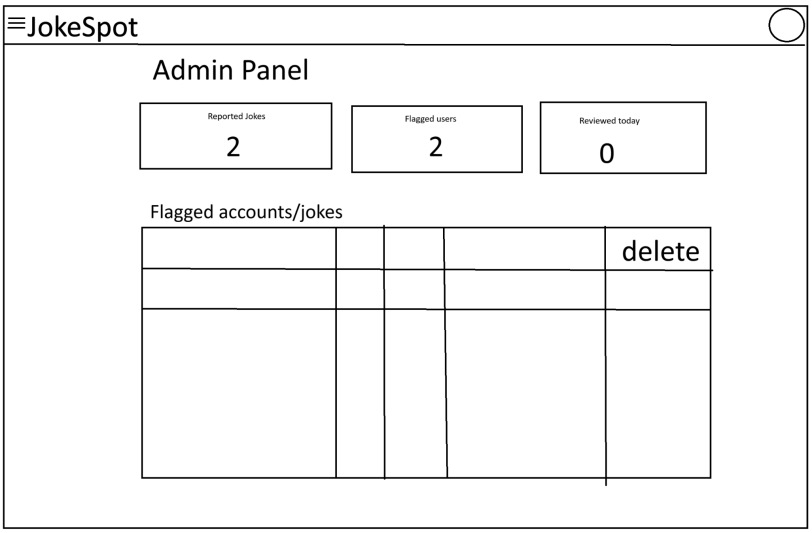
username

******

login

Don't have an account?
Register





Test Results

Overview: these tests were conducted partially inside the development team, though it would have been preferred to test fully outside of the development team. Testing was conducted to ensure the functionality of critical features including posting, browsing the feed, admin controls for deletion of accounts, and cascading deletion of user posts after user deletion

Use Case 1: Posting
- Test Cases
1. Hello World post
2. Empty post submission
3. Excessively long post
4. Special character input
5. Posts containing vulgar content

OBSERVATIONS:
Team typed posts and submitted them to the database, team attempted to submit empty posts as well as posts that exceeded the 500 character limit in the database, team also tested edge cases including special characters and vulgar content. Users were met with confirmation after a successful post and if the cases of overfull or empty posts occurred, were met with a message saying that the post needed to have content or exceeded the post limitation. Users observed that valid posts appeared on the feed immediately after posting, and that the system did not truncate posts that were too long or added content to empty posts; error messages occurred in both scenarios. Messages appear on the post screen to indicate that a post was successful, upon an unsuccessful attempt, the message changes to indicate that the post could not be made. Additionally the vulgar content filter when applied, censors posts on the feed page when a post is successful.

Use Case 2: Scrolling the feed
- Test Cases
1. Standard feed loading
2. Continuous Scrolling
3. End of Feed Reached
4. Empty Feed Handling

OBSERVATIONS:
The user entered the feed page and began reading posts, and then they pressed the button below the feed to load more jokes. They then went to their settings page and turned on the autorefresh feature to load jokes automatically. Back in the feed page, the user scrolled down through jokes, awaiting the next batch to load. Upon reaching the end of the jokes, a message in the feed appeared stating “no more jokes.” Afterwards, the user was asked to search for jokes containing the word “glob,” however, there were no jokes containing “glob” in the content. This resulted in an empty feed where a “no more jokes” message appeared in the feed.

Use Case 3: Admin Deletion of Accounts
- Test Cases
1. Superuser can access control panel
    - Normal users cannot access the /admin page (403 forbidden error)
2. Superuser can delete user accounts through admin panel (with confirmation prompt and success confirmation)

OBSERVATIONS:
When logging as the Superuser (user0) they were able to consistently access the access control panel. All other users in the test were unable to see any sort of button or link that could take them to the admin panel and if they tried to reach the page by modifying the link with /admin they would be redirected to a page displaying a 403 forbidden error message. Superuser was able to delete jokes without issue and would successfully remove the joke from the database, as well as revoking any likes on it related to the account. Superuser could also delete accounts, which would cause all of their jokes to be deleted and the user rendered unable to log into their account. We did observe that if the Superuser attempts to delete an account that is actively

logged in on our Render site, the deletion will not occur, and the user in question can continue using the website.

Use Case 4: Cascading Deletion of Posts after Account Deletion
- Test Cases
1. Deleted users cannot log back into account after deletion
2. Upon account deletion, all user posts are also deleted

OBSERVATIONS:
Watching the user showed that some of our buttons and UI labels weren't as descriptive or explained what each page/purpose of certain elements was. The admin uses the panel to confirm and delete a user. After account deletion, the user could no longer sign-in to their account. The behavior of the website deleting a user account while they are logged in is still in question. Users did happen to get logged off when multiple accounts were active at the same time but we think this might be a render issue. Thus, the behavior was consistent with the use case, except that when a user is logged in and deleted, their account remains in session for them which could pose problems. Once a user account is deleted, they may still register with the same credentials again, although account data is completely reset. Jokes the deleted user posted disappeared from the feed and their account no longer appeared in the leaderboard. The main deviation from what the unauthenticated user was told to do as a test was just that he was having fun exploring the site and testing out what he could do in general. All he needed to do was post a few jokes and report some others. From his behavior we learned that:

- Overfilling the post joke input crashes the user.
- A long joke doesn't text-wrap in the admin panel column, making the table very wide and hard to use.
- Flagged users remain flagged even after all their jokes are cleared.

This testing provided exposure to a lot of unusual behavior and oversights within our application. However, since we lack the time and resources required to provide more support and updates for the project, we have decided to leave it as is.

Deployment
Deployed Site: https://jokespot.onrender.com/

This application was deployed to render and can be accessed at the link above. The front end, back end, and database were deployed to the site which makes the application fully functional just by accessing the site. Alternatively, Docker can be used to create an image for the application to run, however the database is reset with every image.